Study guide: hyperedge percolation project

Read this once end to end tonight. Then read section 1 out loud three times without looking.

Note before you start: slide 12 describes the concentration rule, and the coverage rule was the failed first version.

---

1. The one-minute version

Say these without notes.

1. Many real groups are hyperedges. A committee is one set of members, not a pile of pairs.
2. The usual shortcut flattens a hypergraph into a graph. That over-counts big groups and forgets that the pairs came from one event.
3. Kovács, Benedek and Palla (2025) adapt clique percolation to hypergraphs. They build communities from hyperedges directly.
4. They give two variants. One keeps large hyperedges. The other keeps small hyperedges.
5. The paper sets the parameters by hand, tests on a clean size cut, and compares to clique percolation by picture only.
6. We re-implemented both variants with tests and checked them in the paper's synthetic setting. We do not claim to reproduce its figures.
7. We measured what happens when the size cut is not clean. Percolation works exactly as long as one hyperedge size is pure. When no size is pure, it degrades fast.
8. Clique percolation on the projection often cannot finish at all on large community hyperedges. It times out. On the 6-8 layout all 70 of its runs timed out.
9. We tried a label-free rule to pick parameters. It scores 0.555 and 0.586 for the two variants. The paper default scores 0.292 and 0.229. The oracle scores 0.644 and 0.698.
10. On real committee data, party labels are a weak test. We report that

as a limit, not a success.

---

2. Concepts

Hypergraph and hyperedge size

A hyperedge is a set of any number of nodes. Its size is how many nodes it holds. Example: {a,b,c,d} is one hyperedge of size 4. {a,b} is a hyperedge of size 2, which is an ordinary edge. Say: "A graph edge joins two nodes. A hyperedge joins any number."

Clique projection and what it loses

The projection replaces each hyperedge by all pairs inside it. {a,b,c,d} becomes ab, ac, ad, bc, bd, cd. That is 4*3/2 = 6 pairs. Size s gives s(s-1)/2 pairs. Two losses. Big groups over-count (size 8 gives 28 pairs, size 3 gives 3). And the graph cannot tell {a,b,c,d} from six separate meetings. Say: "Flattening turns one event into many pairs and forgets they belong together."

Community

A set of nodes with many connections inside and few outside. In a hypergraph, "connection" means shared hyperedges.

Partition versus cover

A partition puts every node in exactly one community. A cover lets communities overlap and lets some nodes sit in none. Example on a,b,c,d,e: partition {a,b},{c,d,e}. Cover {a,b,c},{c,d} with e unassigned. Node c overlaps. Say: "Percolation returns a cover, so overlap and unassigned nodes are possible."

k-clique percolation (Palla 2005)

A k-clique is k nodes all linked to each other. Two k-cliques are adjacent if they share k-1 nodes. A community is a chain of adjacent k-cliques, merged into one node set. Example, k=3. Triangles abc and bcd share b,c (2 = k-1), so they are adjacent. Together they form community {a,b,c,d}. Triangle def shares only d with bcd, so it is not adjacent. Nodes in no triangle are left

out. The parameter k sets how tight a group must be. Say: "Roll a k-clique across neighbours that share k-1 nodes. Everything it touches is one community."

Hyperedge percolation, variant 1 (large edges)

Keep hyperedges of size at least k. Two are adjacent if they share at least I_abs nodes, with I_abs from 1 to k-1. A community is a connected chain of them. I_abs = k-1 is the classical clique rule. Example, k=3, I_abs=2. {a,b,c} and {b,c,d} share 2, so they join. {d,e,f} shares one node with {b,c,d}, so it stays out. {a,b} is size 2 and is dropped. Say: "Drop small edges, then chain big edges that overlap enough."

Hyperedge percolation, variant 2 (small edges)

Keep hyperedges of size at most K. A hyperedge of size S joins a growing community if at least floor(I_rel * S) of its nodes are already in it. I_rel is in [0.5, 1). Example, I_rel=0.5. Community {a,b,c,d}. Hyperedge {c,d,e,f} has S=4, needs 2, has 2, so it joins and e,f enter. Hyperedge {d,x,y,z} needs 2, has 1, stays out. Size 2 edges always join. floor(0.5*2)=1 and any edge touching the community has at least one node in it. That is why the paper drops pairwise edges on real data. Say: "Drop huge edges, then let an edge join when half or more of it is already inside."

The random modular hypergraph generator

N=500 nodes, C=10 communities of 50. For each intra size, each community gets N/C = 50 hyperedges drawn inside the community. For each noise size, round(N*p) hyperedges are drawn from all nodes. p is the noise fraction, 0 to 0.4 in the paper. The paper has two types. Large type: communities from sizes 6,7,8, noise 3,4,5. Small type: the mirror. Say: "Planted communities plus random cross-community noise, with sizes we control."

Element-centric similarity (ECS)

It compares two clusterings node by node. For each node it builds an affinity vector over all nodes using personalised PageRank (alpha 0.9) on the node-community structure. In a partition, nodes in the same community are "close". ECS asks whether each node has the same close neighbours in both clusterings, then averages. It works with overlaps and

unassigned nodes. Unassigned nodes count as singleton communities. 1 means identical. Toy fact. 100 nodes, a perfect split into two halves versus all singletons scores about 1/50 = 0.02. Near 0. Say: "ECS asks, for every node, whether its neighbourhood in the clustering is the same in both."

Coverage

Fraction of nodes in at least one community. Unassigned nodes lower it.

Largest-community fraction

Size of the biggest community divided by N.

Giant-community rule

Reject a setting if the largest community is more than twice the second largest. It is the classical clique percolation sign that everything merged.

Concentration score (our label-free rule)

Each node is split equally across the communities that contain it. A node in m communities contributes 1/m to each. A community's effective size is the sum of those contributions. The score is the sum of squared effective sizes divided by n squared. Survivors are settings with at least two communities and a largest community at most twice the second largest. The highest score among survivors wins. Ties go to the stricter setting. If nothing survives, we use the paper default. Say: "A few big communities score high. Many small overlapping ones score low."

---

3. What the paper claims and how it tested it

Claims. Hyperedge percolation is the natural hypergraph version of clique percolation. Its communities can differ from clique percolation on the projection. The small-edge variant suits data where big edges are weak ties.

Tests. - Figure 1 heat maps. 500 nodes, 10 communities, ECS to the planted partition, averaged over 100 realisations. Each panel varies the parameters (k and I_abs, or K and I_rel). The top row has no noise. The bottom row adds 50 hyperedges of each of three noise sizes. - The key design point. The noise sizes never overlap the community sizes. Large noise can be filtered

by size, so the right filter is obvious. Their words: noise "can be completely filtered out". - Qualitative tests on a hyperbolic hypergraph model and two real datasets, with no ground truth. - Clique percolation on the projection is compared by picture only.

---

4. The three gaps we address

1. Parameters are hand-set. A real user has no labels. So we need a way to choose.
2. The synthetic test has a clean size cut. Real data does not. So we let community and noise sizes overlap.
3. The clique comparison is by example. So we measured it over many instances.

---

5. Our experiments

E1: check in the paper's setting (30 seeds, final)

Asks: does our code behave as the paper describes? Figure is a heat map per variant, ECS over parameters, one panel per noise level.

Large type (intra 6-8, noise 3-5). - Large-edge variant: ECS 1.000 at every noise from 0 to 0.4. Under noise the best setting is k=6, I_abs=3. - Small-edge variant: 1.000 at noise 0, then 0.711, 0.571, 0.482, 0.424 at noise 0.1 to 0.4.

Small type (intra 3-5, noise 6-8). - Small-edge variant: 1.000 at all noise (K=5). - Large-edge variant: 0.400, 0.282, 0.233, 0.206 at noise 0.1 to 0.4.

At noise 0 both variants recover the planted communities exactly. Heat map detail, matching the paper's remark: for the large-edge variant, I_abs of 4 or more fragments the communities (ECS about 0.86 to 0.91) even at noise 0. Size 6 to 8 hyperedges rarely share 4 or more nodes, so the chain breaks. Say: "The right variant stays perfect. The wrong one collapses. That is the paper's story, reproduced in its setting."

E2: unclean size cuts (30 seeds)

Asks: what if community and noise sizes overlap? Each method is scored at

its oracle-best parameters. Figures: ECS map and coverage map, one panel per layout, noise 0 to 0.8.

Five layouts. - Clean cuts (6-8 vs 3-5, and 3-5 vs 6-8): the matching variant stays at 1.000 up to noise 0.8. - Partial overlap, intra 5-7 vs noise 4-6: large-edge variant stays at 0.999 (1.000 at noise 0), because k=7 keeps only size 7, which is pure. - Partial overlap, intra 4-6 vs noise 5-7: small-edge variant stays about 0.97, because K=4 keeps only size 4, which is pure. - Full overlap, 4-6 vs 4-6: both collapse together. 1.000 at noise 0, about 0.60 at 0.1, 0.46 at 0.2, 0.38 at 0.3, 0.34 at 0.4, 0.27 at 0.6, 0.24 at 0.8.

Message. Percolation works exactly as long as at least one hyperedge size is pure. With no pure size it degrades fast, and the two variants become the same.

Clique baseline. On the large-community type, the projection of a 50-node community built from 150 hyperedges of size 6 to 8 is nearly a complete graph. Enumerating its cliques does not finish within 60 s or runs out of memory. We ran k=3 and k=4 only, 35 runs each per layout. Timed-out runs are left out of the means. - Intra 6-8 vs noise 3-5: all 35 runs at k=3 and all 35 at k=4 failed (1.000). Intra 5-7 vs 4-6 is the same. - Intra 4-6 vs 4-6: 0.971 failed at k=3, 0.829 at k=4. - Intra 4-6 vs 5-7: 0.971 failed at k=3, 0.886 at k=4. - Intra 3-5 vs 6-8: 0.143 failed at both k. - Where it finished, clique percolation scores 1.000 at noise 0 and 0.100 to 0.105 once noise is added. It has no score at all in most cells above noise 0.2. Say: "Most layouts time out almost every run." That is a finding about cost, not a scoring result.

E3: label-free rule (30 seeds, final)

Asks: can we pick parameters without labels, and how far from the best?

The rule, in steps. 1. Sweep the parameter grid. 2. For each setting, record the community sizes. 3. Reject if there are fewer than two communities. 4. Reject if the largest is more than twice the second largest. 5. Score the rest by concentration. Each node splits its weight equally across its communities, so a node in m communities gives 1/m to each. A community's effective size is the sum of those shares. Score = sum of squared effective sizes, divided by n squared. 6. Pick the highest score. Ties go to the stricter setting. 7. If nothing survives, use the paper default.

Why coverage failed. A bad setting can produce many tiny overlapping fragments that together touch every node. Coverage is then near 1, so coverage rewards fragmentation. Squaring a concentrated mass rewards a few big communities. Splitting each node across its communities first stops overlap from inflating the score. The final numbers back this only in part. In mean ECS the two rules are close. But the coverage rule reaches the oracle far less often. It does better only when the variant does not match a pure size, and under full overlap.

Numbers, averaged over all settings and noise levels (1050 instances per variant). The oracle here is the best setting for each single hypergraph. - Large-edge variant: rule 0.555, coverage-only rule 0.547, oracle 0.644, default 0.292. Reaches the oracle on 46.6 percent of instances (coverage rule 17.8 percent). - Small-edge variant: rule 0.586, coverage-only 0.544, oracle 0.698, default 0.229. Reaches the oracle on 55.4 percent (coverage rule 21.9 percent).

Per setting (rule against oracle, mean over noise). - Matching variant with a pure size: large-edge on 6-8 vs 3-5 gets 0.936 against 1.000. Large-edge on 5-7 vs 4-6 gets 0.983 against 0.999. Small-edge on 3-5 vs 6-8 gets 1.000 against 1.000. Small-edge on 4-6 vs 5-7 gets 0.974 against 0.974. Hit rates are 91.9, 98.1, 100 and 100 percent. - Wrong variant: small-edge on 6-8 vs 3-5 gets 0.263 against 0.548. Small-edge on 5-7 vs 4-6 gets 0.286 against 0.493. Large-edge on 3-5 vs 6-8 gets 0.233 against 0.351. Large-edge on 4-6 vs 5-7 gets 0.226 against 0.399. Hit rate is 14.3 percent in each, which is the noise 0 case only. - Full overlap, 4-6 vs 4-6: large-edge 0.396 against 0.471, small-edge 0.410 against 0.476. Hit rates are 14.3 and 48.6 percent. The gap is small because the oracle itself is low.

Reading the figure. Big win over the default, clear gap to the oracle. The rule is near the oracle when the variant matches a pure size. It is weakest when the variant is the wrong one, where the gap reaches 0.285.

E4: real committees (final)

House: 1290 nodes, 341 hyperedges, mean size about 35, two party labels. Senate: 282 nodes, 315 hyperedges, mean size 17. Pairs are dropped. Best ECS against party is 0.501 for the House and 0.500 for the Senate. Each comes from one community containing everyone, since ECS of a single

community versus a 50/50 split is about 0.5. So party is not what committee co-membership percolation finds. Each line gives ECS to party, number of communities, coverage. - House, large-edge: rule 0.501, 1, 1.000. Oracle 0.501, 1, 1.000. Default 0.501, 1, 1.000. - House, small-edge: rule 0.026, 6, 0.156. Oracle 0.501, 1, 1.000. Default 0.501, 1, 1.000. - Senate, large-edge: rule 0.488, 18, 1.000. Oracle 0.500, 1, 1.000. Default 0.485, 2, 1.000. - Senate, small-edge: rule 0.194, 13, 0.525. Oracle 0.485, 2, 1.000. Default 0.485, 2, 1.000. House, large-edge detail. No setting in the grid passes the giant-community test: each gives either one community or a giant community more than twice the next one. So nothing survives, and the rule's pick (k=3, I_abs=2) is the fallback to the paper default, not a selection. Honest reading. The labels are a weak test. The rule does not help here. For the small-edge variant its pick scores far below the default and leaves nodes unassigned. This is a limit, not a success.

---

6. Limits to state before anyone asks

- Synthetic communities are disjoint, so overlap is never tested.
- Two-party labels are weak ground truth.
- Percolation leaves nodes unassigned, and ECS penalises that.
- The rule is a heuristic. The giant rule assumes communities of similar size.
- The clique baseline is limited by time-outs, not by accuracy.
- On real committees the rule is no better than the default, and worse for the small-edge variant.

---

7. Fifteen likely questions

1. **Why not modularity?** Modularity is defined on graphs and assumes a partition. We want covers and we want to avoid flattening. Also, we have planted labels, so comparing to them is more direct.
2. **Why ECS and not NMI?** NMI needs a partition. ECS handles overlap and unassigned nodes, and the paper uses it.
3. **How is this different from clique percolation?** Clique percolation runs on pairs. Ours runs on hyperedges directly, so a hyperedge

stays one event and big hyperedges are not blown up into s(s-1)/2 pairs.

4. **What does it cost?** We only compare hyperedges, not every clique. In our runs it finished. The projection baseline did not, on large hyperedges. We did not do a formal complexity study.
5. **Why 500 nodes?** It is the paper's setting. Same N, same 10 communities, so our checks are comparable.
6. **What does the default parameter mean?** I_abs = k-1 with k the smallest size at least 3, or K the largest size with I_rel = 0.5. It is the classical clique rule, with no tuning.
7. **Is 1.0 suspicious?** No. At noise 0 the planted structure is literally the only structure. With a pure size, noise is filtered out by size, so it stays at 1.
8. **What happens on real data?** Party labels give at best 0.501 for the House and 0.500 for the Senate, which is the trivial value. If asked why the rule picked one community on the House: it did not. Nothing survived the giant rule, so it fell back to the default. Percolation finds structure, but not party. We say the test is weak.
9. **Why drop pairwise edges?** A size-2 edge always joins a community in the small-edge variant. It would merge everything. The paper drops them, so we do too.
10. **What next?** Overlapping planted communities, labels richer than two parties, and a better rule for communities of different sizes.
11. **How does the rule know there is no label?** It never looks at labels. It uses only community sizes from the output and the structure of the result.
12. **Is the generator realistic?** No. It is a controlled test with a known answer. Real groups overlap and vary in size.
13. **Why both variants?** They assume opposite things. One says big edges carry structure, the other says small edges do. Our E1 and E2 results show each wins when its assumption holds.
14. **Did you reproduce the paper?** We re-implemented it and checked behaviour in its setting. We do not claim to reproduce its figures. We use 30 seeds, the paper uses 100.
15. **What does noise 0.4 mean?** round(500*0.4) = 200 noise hyperedges per noise size. With three sizes that is 600 noise hyperedges, against 1500 community hyperedges (10 communities, 50 per size, 3 sizes).

8. Slide-by-slide script

1. Title. This is a re-implementation of a 2025 paper on community detection in hypergraphs. We add a measured comparison and a rule to pick parameters. We make no claim to reproduce the paper's figures.

2. Why hypergraphs. A graph edge joins two nodes. A hyperedge joins any number. A class of 8 children in contact is one hyperedge of size 8, not 28 pairs. Flattening forgets that the pairs came from one event and over-counts big groups.

3. Communities, partitions and covers. A community is a set with many links inside and few outside. A partition gives each node one community. A cover allows overlap and unassigned nodes. Percolation methods return covers.

4. Clique percolation. A k-clique is k nodes all linked. Two k-cliques are adjacent if they share k-1 nodes. A community is a chain of adjacent cliques. It can overlap and it leaves out nodes in no clique.

5. The paper's idea. Skip the graph. Run percolation on hyperedges directly. Two hyperedges are adjacent if they share enough nodes. A community is everything reachable by a chain.

6. Variant 1, large hyperedges. Keep hyperedges of size at least k. Join two if they share at least I_abs nodes. I_abs = k-1 is the classical rule. Use it when many tiny edges are noise.

7. Variant 2, small hyperedges. Keep hyperedges of size at most K. An edge of size S joins if at least floor(I_rel * S) of its nodes are inside. Use it when huge edges are weak ties, like an assembly.

8. How the paper tests it. 500 nodes, 10 planted communities of 50. Community edges have sizes 6-8 or 3-5. Noise has the other sizes and is drawn from all nodes, up to 40 percent. Score is ECS. They also use a hyperbolic model and two real datasets with no ground truth.

9. Element-centric similarity. It compares two clusterings node by node using personalised PageRank. It handles overlap and unassigned nodes,

which count as singletons. 1 means identical. A two-halves split against all singletons scores about 0.02 on 100 nodes.

10. What the paper leaves open. Parameters are hand-set. The synthetic test has a clean size cut, so the right filter is obvious. The clique comparison is by example, not measured.

11. What we did. We re-implemented both variants and the generator, with tests. E1 checks the paper's setting. E2 measures hyperedge versus clique percolation with unclean cuts. E3 proposes a label-free rule. E4 tries real committee data.

12. Our rule. Sweep the parameters. Reject settings with fewer than two communities, or where the largest is over twice the second. Among the rest, pick the highest concentration score, with ties to the stricter setting. Coverage was the first version and it failed, because tiny overlapping fragments give coverage near 1.

13. E1, large type. Same setting as the paper's Figure 1. The large-edge variant stays at 1.000 for all noise. The small-edge variant falls from 1.000 to 0.424. At noise 0 both are exact. I_abs of 4 or more fragments the communities, as the paper notes.

14. E1, small type. The mirror. The small-edge variant stays at 1.000. The large-edge variant falls to 0.206. The wrong variant fails, the right one does not.

15. E2 map. When one hyperedge size is pure, the matching variant survives, even at noise 0.8. When no size is pure, both collapse to about 0.24 at noise 0.8. Clique percolation fails on all 70 runs at k=3 and k=4 when community edges have sizes 6-8. Where it finishes, it scores about 0.10 once noise is added.

16. E2 coverage. Coverage stays near 1 for the hyperedge variants. So low ECS comes from wrong merges and splits, not from unassigned nodes. Say this only if the figure shows it.

17. E3. Rule, oracle and default. Large-edge variant: rule 0.555, oracle 0.644, default 0.292. Small-edge variant: rule 0.586, oracle 0.698, default 0.229. The rule hits the oracle on 46.6 and 55.4 percent of instances. It is near the

oracle when the variant matches a pure size. It is weakest when the variant is the wrong one.

18. E4. House has 1290 nodes and 341 hyperedges. Senate has 282 nodes and 315. Best ECS against party is 0.501 for the House and 0.500 for the Senate, from one community of everyone. That equals a trivial score, so party labels are a weak test. On the House with the large-edge variant, no setting passes the giant-community test, so nothing survives and the rule falls back to the default. The rule's pick scores 0.501 and 0.488 for the large-edge variant. For the small-edge variant it scores 0.026 and 0.194, with 6 and 13 communities and coverage 0.156 and 0.525. The default scores 0.501 for the House and 0.485 for the Senate.

19. What we found. Four points. The right variant is exact when a size is pure. With no pure size, both fail together. Clique percolation is too costly on big hyperedges. The rule beats the default but trails the oracle.

20. Limits. Synthetic communities are disjoint. Real labels are two parties. Percolation leaves nodes unassigned. The rule is a heuristic, and its giant test assumes similar community sizes.

21. Conclusion. Hyperedge percolation behaves as the paper says in its setting. It works exactly when one size is pure and degrades otherwise. Our rule is a cheap improvement over the default, with a clear gap to the oracle.